内 容 安 排

章	内 容	学时	章	内 容	学时
1	绪 论	2	7	图	5
2	线性表	5	8	动态存储管理	略
3	栈和队列	3	9	查找	3
4	串	3	10	内部排序	5
5	数组和广义表	3	11	外部排序	略
6	树和二叉树	7	12	文件	略

第3章小结

线性表、栈、队的异同点：

相同点：**逻辑结构相同，都是线性的；都可以用顺序存储或链表存储；栈和队列是两种特殊的线性表，即受限的线性表（只是对插入、删除运算加以限制）。**

不同点：① **运算规则不同：**

- **线性表为随机存取；**
- **而栈是只允许在一端进行插入和删除运算，因而是后进先出表LIFO；**
- **队列是只允许在一端进行插入、另一端进行删除运算，因而是先进先出表FIFO。**

② **用途不同。线性表比较通用；堆栈用于函数调用、递归和简化设计等；队列用于离散事件模拟、OS作业调度和简化设计等。(以后再举例)**

第4章　串（String）

4.1　串类型的定义

4.2　串的表示和实现

4.3　串的模式匹配算法

主要内容

串
- 逻辑结构　$s = ‘a_1a_2 \ldots\ldots a_n’$
- 存储结构
  - 定长顺序存储结构
  - 堆存储结构
  - 块链存储结构
- 操作(或运算)
  - 若干函数的实现
  - 模式匹配算法

模式匹配即子串定位运算，即如何实现 **Index(S,T,pos)**函数

精华：**KMP**算法——快速（用**next[j]**或**nextval[j]**)

4.1　串类型的定义

串即字符串，是由零个或多个字符组成的有限序列，是数据元素为单个字符的特殊线性表。

记为：　　$s = ‘a_1 a_2 \ldots\ldots\ldots a_n’$　　$(n \geq 0)$

串名　串值（用‘’括起来）

隐含结束符‘\0’，即ASCII码NUL

为何要单独讨论“串”类型？

1）字符串操作比其他数据类型更复杂（如拷贝、连接操作）

2）程序设计中，处理对象很多都是串类型。

若干术语：

串长：串中字符的个数（n≥0）。 n=0 时称为空串 ∅ 。

空白串：由一个或多个空格符组成的串。

问：空串和空白串有无区别？

答：有区别。

空串(Null String)是指长度为零的串；

而空白串(Blank String),是指包含一个或多个空白字符‘ ’(空格键)的字符串.

子串：串S中任意个连续的字符序列叫S的子串; S叫主串。

子串位置：子串的第一个字符在主串中的序号。

字符位置：字符在串中的序号。

串相等：串长度相等，且对应位置上字符相等。

例1：现有以下4个字符串：

a =‘BEI’　b =‘JING’　c = ‘BEIJING’　d = ‘BEI JING’

问：① 他们各自的长度？　a =3，b =4，c = 7，d=8

② a是哪个串的子串？在主串中的位置是多少？

a是c和d的子串，在c和d中的位置都是1

③ 空串是哪个串的子串？ a是不是自己的子串？

“空串是任意串的子串；任意串S都是S本身的子串，除S本身外，S的其他子串称为S的真子串。”

——《数据结构与算法》中山大学出版社

串的抽象数据类型定义（参见教材P71）

ADT String{

Objects: $D=\{a_i \mid a_i \in CharacterSet, i=1, 2, \ldots, n, n \geq 0\}$

Relations: $R1=\{<a_{i-1},a_i> \mid a_{i-1},a_i \in D, i=2, \ldots, n\}$

functions: //至少有13种基本操作

最小操作子集

StrAssign(&T, chars) // 串赋值，生成值为chars的串T

StrCompare(S,T) // 串比较，若S>T，返回值大于0…

StrLength(S) // 求串长，即返回串S中的元素个数

Concat(&T, S1, S2) // 串连接，用T返回S1 + S2的新串

SubString(&Sub, S, pos, len) // 求S中pos起长度为len的子串

……

int Index(S, T, pos) //子串定位函数（模式匹配），返回位置

Replace(&S, T,V) // 用子串V替换子串T

}ADT String

C语言中已有类似的串运算函数！

复习：C语言中常用的串运算

注：用C处理字符串时，要调用标准库函数 #include<string.h>

	C	类C
串比较：	int strcmp(char *s1,char *s2);	StrCompare(S,T)
求串长：	int strlen(char *s);	StrLength(S)
串连接：	char strcat(char *to,char *from)	Concat(&T, S1, S2)
子串T定位：	char strchr(char *s,char *c);	Index(S, T, pos)

……

注：Concat操作＝concatenation，把多个短字符串合并为长字符串

例1： 设 s ='I AM A STUDENT', t ='GOOD', q='WORKER'。求： (参见P71)

StrLength(s) ＝ 14

StrLength(t) ＝ 4

SubString(&sub, s, 8, 7)= 'STUDENT'

SubString(&sub, t, 2, 1)='O'

Index(s, 'A')= 3

Index(s, t)= 0 （s中没有t='GOOD'）

Replace(&s, 'STUDENT', q)= 'I AM A WORKER'

Index(S, T, pos)
// 返回子串T在pos之后的位置

Replace(&S, T,V)
// 用子串V替换子串T

提问： 当s ='I AM A STUDENT'时，INDEX（s,'A'，pos）=3，若想搜索后面那个‘A’怎么办？

答：根据教材P71倒1行的函数说明， INDEX（s,'A'）**返回的只是“第一次”出现的位置。**

如果还要搜索后面的A，则pos变量应当跟着变。

也就是说，要把得到的“第一次”位置再代入INDEX（ s,'A'，pos**）函数中循环操作才行。**

例2：设 s ='I AM A STUDENT', t ='GOOD',求：

Concat(SubString(s,6,2), Concat(t,SubString(s,7,8))) ＝?

（摘自严题集4.3 ①）

解：因为

SubString(s,6,2)＝ ‘A’;

SubString(s,7,8)＝ ‘STUDENT’

Concat(,t,SubString(s,7,8))＝’ GOOD STUDENT’

所以：

Concat(SubString(s,6,2), Concat(t,SubString(s,7,8)))

＝ ‘A GOOD STUDENT’

4.2 串的表示和实现

首先强调：串与线性表的运算有所不同，是以“串的整体”作为操作对象，例如查找某子串，在主串某位置上插入一个子串等。

串有三种机内表示方法：

顺序存储

- 定长顺序存储表示

  ——用一组地址连续的存储单元存储串值的字符序列，属静态存储方式。

- 堆分配存储表示

  ——用一组地址连续的存储单元存储串值的字符序列,但存储空间是在程序执行过程中动态分配而得。

链式存储

- 串的块链存储表示

  ——链式方式存储

定长顺序存储特点： 用一组连续的存储单元来存放串，直接使用定长的字符数组来定义，数组的上界预先给出，故称为静态存储分配。

例如 (P73):

```
#define Maxstrlen 255    //用户可用的最大串长
  typedef unsigned char SString[ Maxstrlen + 1 ] SString;
      SString    s;    //s 是一个可容纳255个字符的顺序串。
```

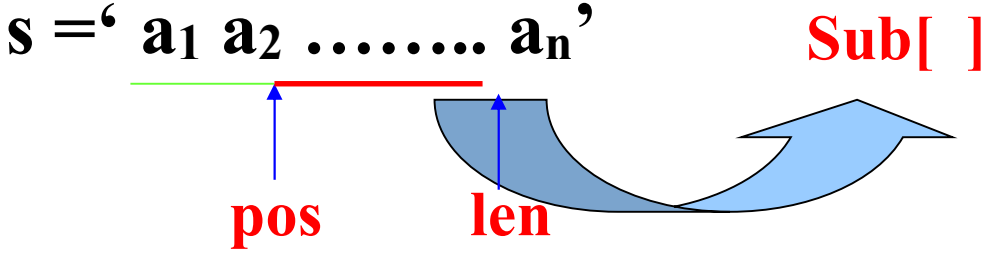
注:

- **一般用SString[0]来存放串长信息（如pascal语言）；**
- **C语言约定在串尾加结束符 ‘\0’，以利操作加速，但不计入串长（用首址和串长、或首址和尾标记来描述串数组）**
- **若字符串超过Maxstrlen 则自动截断（因为静态数组存不 进去）。**

只能记录255B?

例：用顺序存储方式编写求子串函数SubString(&Sub, S, pos, len)

将串S中从第pos个字符开始、长度为len的字符序列复制到串Sub中。
(注：考虑到函数的通用性，应当让串Sub的预留长度与S一样)




```
Status SubString(SString &sub, SString S, int pos, int len )
{  if(pos<1 || pos>S[0] || len<0 || len>S[0]-pos+1) return ERROR;
//若pos和len参数越界，则告警
   Sub[1……len]=S[pos……pos+len-1];
   Sub[0]=len;   return OK;
}
```

子串长度

讨论：想存放超长字符串怎么办?
改用动态分配的一维数组——堆

堆分配存储特点：仍用一组连续的存储单元来存放串，但存储空间是在程序执行过程中动态分配而得。

思路： 利用malloc函数合理预设串长空间。

特点： 若在操作中串值改变，还可以利用realloc函数按新串长度增加空间 **(即动态数组概念)** 。

堆T的存储结构描述：

```
Typedef struct {
        char *ch;     //若非空串,按串长分配空间; 否则ch=NULL
        int length;   //串长度
}HString
```

各分量书写方式：
T.ch
T.length

例1：编写建堆函数 (参见教材P76）

```
Status StrAssign(HString &T, char *chars)
{ //生成一个串T, T值←串常量chars
  if (T.ch)  free(T.ch);    //释放T原有空间
  for (i=0, c=chars;  c;  ++i, ++c);      //求chars的串长度i
        if ( !i ) {T.ch = NULL; T.length = 0;}
        else{
            if (!(T.ch = (char*)malloc( i *sizeof(char))))
            exit(OVERFLOW);
           T.ch[0..i-1] = chars[0..i-1];
           T.length =i;
        }
        Return OK;
 } //StrAssign
```

C是指针变量，可以自增！意即每次后移一个数据单元。

直到终值为“假”停止，串尾特征是 c = ‘\0’ = NULL = 0

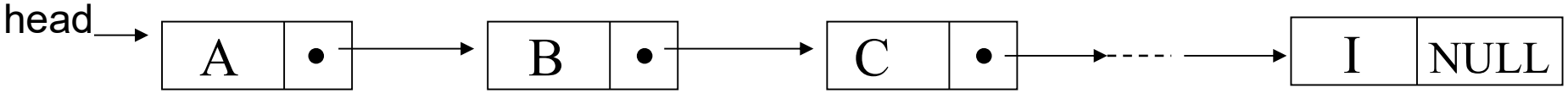
此处T.ch[0]没有用来装串长，因为另有T.length 分量

例2：用“堆”方式编写串插入函数 (参见教材P75)

```
Status StrInsert ( HString &S, int pos, HString T )
{   //在串S的第pos个字符之前（包括尾部）插入串T
if (pos<1||pos>S.length+1) return ERROR;   //pos不合法则告警
  if(T.length){              //只要串T不空，就需要重新分配S空间，以便插入T
 if (!（S.ch=(char*)realloc(S.ch, (S.length+T.length)*sizeof(char)) ))
exit(OVERFLOW);  //若开不了新空间，则退出
      for ( i=S.length-1; i>=pos-1; --i ) S.ch[i+T.length] = S.ch[i];
      // 为插入T而腾出pos之后的位置，即从S的pos位置起全部字符均后移
    S.ch[pos-1…pos+T.length-2] = T.ch[0…T.length-1]; //插入T，略/0
     S.length + = T.length;          //刷新S串长度
} return OK;
} //StrInsert
```

链式存储特点： 用链表存储串值，易插入和删除。

法1：链表结点的数据分量长度取1（个字符）



法2：链表结点（数据域）大小取n(例如n=4)

讨论：法1存储密度为__1/2__；法2存储密度为__9/15=3/5__

显然，若数据元素很多，用法2存储更优—称为块链结构

块链类型定义：

对块链表的整体描述

```
typedef  struct {                    //定义用链式存储的串类型
        Chunk  *head;                //头指针
        Chunk  *tail;                //尾指针
        int  curLen;                 //结点个数
  } Lstring;        //串类型只用一次，前面可以不加结构名称
```

```
#define  CHUNKSIZE  80      //每块大小，可由用户定义
typedef  struct  Chunk {          //首先定义结点类型
      char    ch [ CHUNKSIZE ];   //每个结点中的数据域
      struct  Chunk * next ;      //每个结点中的指针域
}Chunk;
```

对每个结点的描述

块链函数的编写略

4.3　串的模式匹配算法

算法目的：确定主串中所含子串第一次出现的位置（定位）

定位问题称为串的模式匹配，典型函数为Index(S, T, pos)

算法种类：

- **BF算法**　（又称古典的、经典的、朴素的、穷举的）
- **KMP算法**

带回溯，速度慢

避免回溯，匹配速度快，
是全课程的亮点之一

BF算法的实现—即编写**Index(S, T, pos)**函数

例1： S=‘**ababcabcacbab**’，T=‘**abcac**’，pos=1，

求：串T在串S中第pos个字符之后的位置。

BF算法设计思想：

- 将主串S的第pos个字符和模式T的第1个字符比较，
  若相等，继续逐个比较后续字符；
  若不等，从主串S的下一字符（pos+1）起，重新与T第一个字符比较。
- 直到主串S的一个连续子串字符序列与模式T相等。返回值为S中与T匹配的子序列第一个字符的序号，即匹配成功。

否则，匹配失败，返回值 0。

利用演示系统看BF算法执行过程。

第4章作业预告：

严习题集的 **4.8　4.10　4. 30　4.31**共4题

2007百度笔试题：

编程1：找出字符串的最长不重复子串并输出其长度。实现时有无O（n）的算法？

编程2：找出一个字符串的最长回文子串，并使算法效率尽量高。